Hochschule Kempten — Fakultät Informatik

Simulation der Tomasulo-Pipeline

Vorgelegt von: Ceyhun Akca

Inhaltsverzeichnis

Inhaltsverzeichnis 2
1. Einführung 3
2. Struktur und Funktionsweise des Simulators 4
2.1 Architekturübersicht 4
2.2 Komponenten 4
2.3 Phasenreihenfolge pro Zyklus 4
2.4 Getroffene Annahmen 5
3. Validierung des Simulators 6
3.1 Validierung mit dem Referenz-Trace 6
3.2 Weitere Validierungsfälle 6
4. Experimente und Ergebnisse 7
4.1 Experiment A: Tomasulo vs. In-Order-Pipeline 7
4.2 Experiment B: Variation der Reservierungsstationen 9
4.3 Experiment C: Variation der Latenzen 9
4.4 Experiment D: Analyse aufgelöster Abhängigkeiten 9
5. Zusammenfassung und Schlussfolgerungen 11
6. Bedienungsanleitung 11

1. Einführung

Dieser Bericht dokumentiert die Implementierung und Analyse eines Simulators für den Tomasulo-Algorithmus. Der Tomasulo-Algorithmus ist ein dynamisches Scheduling-Verfahren für superskalare Prozessoren, das Out-of-Order-Ausführung durch den Einsatz von Reservierungsstationen und implizitem Register-Renaming ermöglicht.

Der Simulator wurde in Python implementiert und unterstützt die Instruktionen ADD, SUB, MUL, DIV, LOAD und STORE. Er ist modular aufgebaut und erlaubt die Konfiguration aller relevanten Parameter (Anzahl Reservierungsstationen, Latenzen der Funktionseinheiten) über Kommandozeilenargumente.

2. Struktur und Funktionsweise des Simulators

2.1 Architekturübersicht

Der Simulator modelliert eine in-order-issuende, out-of-order-ausführende und out-of-order-abschließende Pipeline. Die folgende Abbildung zeigt die Gesamtarchitektur:

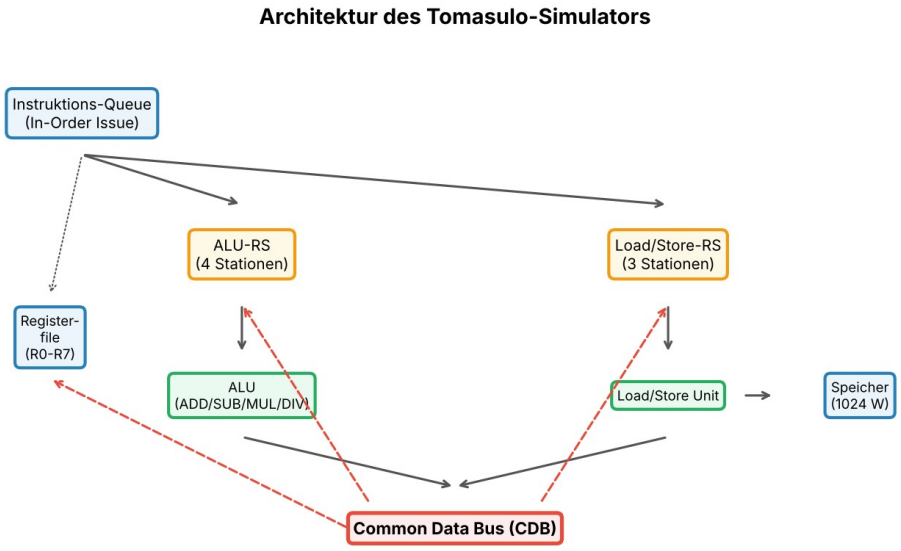


Abbildung 1: Architektur des Tomasulo-Simulators

2.2 Komponenten

Reservierungsstationen (RS): Jede RS speichert die Operation, die Operandenwerte (Vj, Vk) oder Tags der produzierenden RS (Qj, Qk), das Zielregister und den Ausführungsstatus. ALU-Operationen und Load/Store-Operationen verwenden getrennte RS-Pools.

Common Data Bus (CDB): Ein einzelner Bus, über den Ergebnisse an alle wartenden RS und das Registerfile broadcast werden. Pro Zyklus ist maximal ein CDB-Broadcast möglich. Bei Konflikten erhält die älteste Instruktion (frühestes Issue) den Vorrang. STOREs benötigen keinen CDB, da sie keinen Registerwert produzieren.

Register-Renaming: Das Renaming erfolgt implizit über die Reservierungsstationen. Beim Issue wird das Zielregister mit dem Namen der zugewiesenen RS getaggt. Nachfolgende Instruktionen, die dieses Register lesen, erhalten den Tag statt des Wertes. WAR- und WAW-Hazards werden dadurch vollständig eliminiert.

Hauptspeicher: Simuliert als Array mit 1024 Wörtern. Initialisiert mit mem[i] = i × 10 für nachvollziehbare Testergebnisse.

2.3 Phasenreihenfolge pro Zyklus

Jeder Simulationszyklus durchläuft drei Phasen in fester Reihenfolge:

Phase 1 — Write Result (CDB): Fertig ausgeführte Instruktionen broadcasten ihr Ergebnis. STOREs werden separat abgeschlossen (direkter Speicherschreibzugriff). CDB-Werte werden von wartenden RS und dem Registerfile aufgenommen.

Phase 2 — Execute: RS mit allen verfügbaren Operanden beginnen die Ausführung. Bereits laufende Instruktionen werden weiter abgearbeitet (Zyklen-Countdown). Wichtig: RS, deren Operanden erst in Phase 1 desselben Zyklus per CDB eingetroffen sind, dürfen erst im nächsten Zyklus mit der Ausführung beginnen (1-Zyklus-CDB-Verzögerung, gemäß Hennessy & Patterson).

Phase 3 — Issue: Die nächste Instruktion aus dem Trace wird in-order an eine freie RS zugewiesen. Sind alle RS des benötigten Typs belegt, entsteht ein struktureller Stall.

2.4 Getroffene Annahmen

Folgende Annahmen wurden bei der Implementierung getroffen:

1) Maximal eine Instruktion pro Zyklus kann issued werden (single-issue).

2) CDB-Verzögerung: Ein Wert, der per CDB eintrifft, steht erst im nächsten Zyklus für die Ausführung bereit. Dies modelliert die reale Registerverzögerung.

3) CDB-Arbitrierung: Bei gleichzeitiger Fertigstellung mehrerer Instruktionen erhält die älteste (frühestes Issue) den CDB.

4) Keine Memory-Disambiguation: LOADs prüfen nicht, ob vorhergehende STOREs auf dieselbe Adresse ausstehend sind. Dies ist eine Vereinfachung gegenüber realer Hardware.

5) Registerinitialisierung: Ri = i (für einfache Verifikation). Speicherinitialisierung: mem[i] = i × 10.

6) Division durch Null ergibt 0 (kein Trap/Exception).

3. Validierung des Simulators

Zur Validierung wurden fünf Trace-Dateien erstellt, die jeweils einen spezifischen Aspekt des Tomasulo-Algorithmus testen. Für jeden Fall werden die erwarteten und tatsächlichen Ergebnisse verglichen.

3.1 Validierung mit dem Referenz-Trace

Der Referenz-Trace enthält fünf Instruktionen mit verschiedenen Abhängigkeiten: ADD R1,R2,R3 → MUL R4,R1,R5 (RAW auf R1) → LOAD R6,4(R2) → SUB R7,R6,R4 (RAW auf R6 und R4) → DIV R0,R7,R1 (RAW auf R7).

Nr	Instruktion	Issue	Exec	Write
0	ADD R1, R2, R3	1	2	3
1	MUL R4, R1, R5	2	4	7
2	LOAD R6, 4(R2)	3	4	6
3	SUB R7, R6, R4	4	8	9
4	DIV R0, R7, R1	5	10	15

Tabelle 1: Instruktions-Timeline des Referenz-Traces (15 Zyklen, IPC = 0.3333)

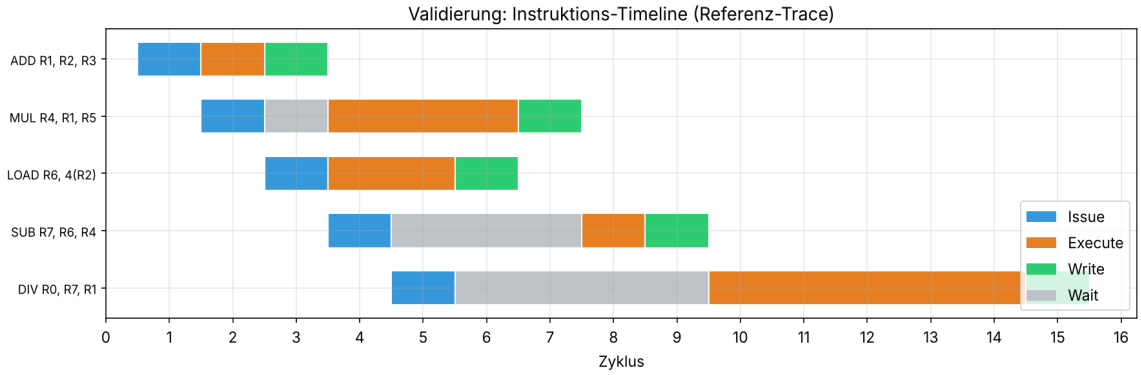


Abbildung 2: Gantt-Diagramm der Pipeline-Ausführung (Referenz-Trace)

Erläuterung: MUL wartet auf R1 (CDB-Broadcast in Zyklus 3) und startet gemäß der CDB-Verzögerung erst in Zyklus 4. SUB benötigt sowohl R6 (Zyklus 6) als auch R4 (Zyklus 7) und startet erst in Zyklus 8. Insgesamt werden 4 RAW-Abhängigkeiten transparent über den CDB aufgelöst.

3.2 Weitere Validierungsfälle

Fall 1 — Keine Abhängigkeiten: 4 unabhängige ADD/SUB-Instruktionen. Ergebnis: 6 Zyklen, IPC=0.6667, 0 Stalls, 0 RAW-Auflösungen. Jede Instruktion durchläuft Issue→Exec→Write ohne Wartezeit — korrekt.

Fall 2 — RAW-Kette (ADD→MUL→SUB): Ergebnis: 9 Zyklen (vs. 11 In-Order), IPC=0.3333, 2 RAW-Auflösungen. Die Kette erzwingt sequentielle Ausführung, aber Tomasulo spart 2 Zyklen durch früheres Issue (Speedup 1.22×).

Fall 3 — Struktureller Stall (5 DIVs, 4 ALU-RS): Ergebnis: 15 Zyklen, 2 Stall-Zyklen. Die 5. DIV-Instruktion muss warten, bis in Zyklus 7 die erste RS freigibt — korrekt.

Fall 4 — WAR/WAW-Auflösung: ADD und SUB schreiben beide R1 (WAW). MUL liest R1 bevor SUB es überschreibt (WAR). Ergebnis: MUL erhält korrekt ADD's R1=5 über den CDB-Tag, unabhängig von SUB's späterer Überschreibung. DIV erhält korrekt SUB's R1=-1. WAR und WAW werden durch Register-Renaming vollständig aufgelöst.

Fall 5 — LOAD/STORE mit CDB: LOAD→ADD→STORE-Kette. Ergebnis: 9 Zyklen, 2 RAW-Auflösungen. LOAD-Ergebnis wird korrekt über CDB an ADD weitergeleitet. STORE wartet auf ADD-Ergebnis und schreibt anschließend in den Speicher.

4. Experimente und Ergebnisse

Die folgenden Experimente untersuchen das Verhalten des Tomasulo-Algorithmus unter verschiedenen Konfigurationen. Alle Experimente verwenden die Standardkonfiguration (4 ALU-RS, 3 LS-RS, ADD/SUB=1, MUL=3, DIV=5, LOAD/STORE=2) als Baseline, sofern nicht anders angegeben.

4.1 Experiment A: Tomasulo vs. In-Order-Pipeline

Verglichen werden die Gesamtzyklen und der IPC-Wert des Tomasulo-Simulators mit einer einfachen In-Order-Pipeline ohne dynamisches Scheduling.

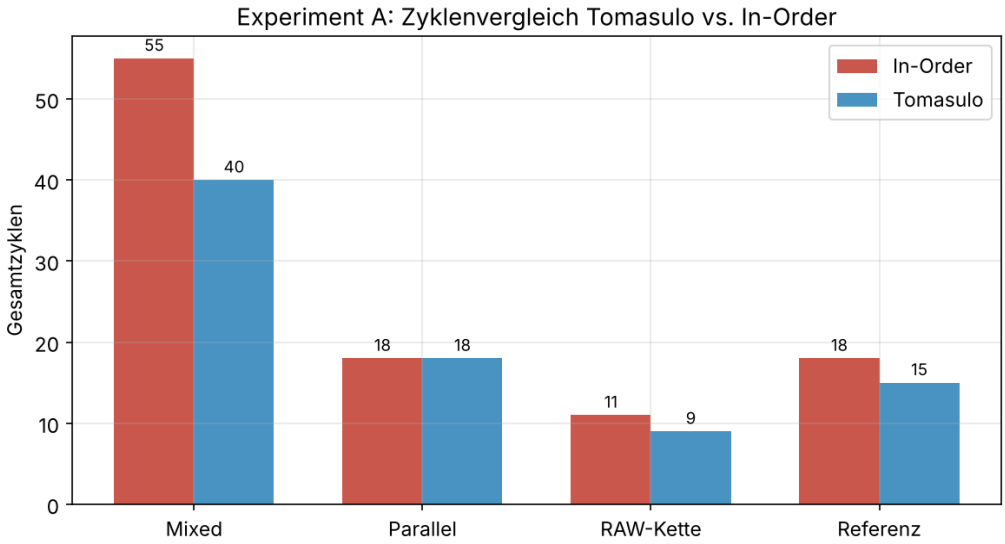


Abbildung 3: Zyklenvergleich Tomasulo vs. In-Order über vier Traces

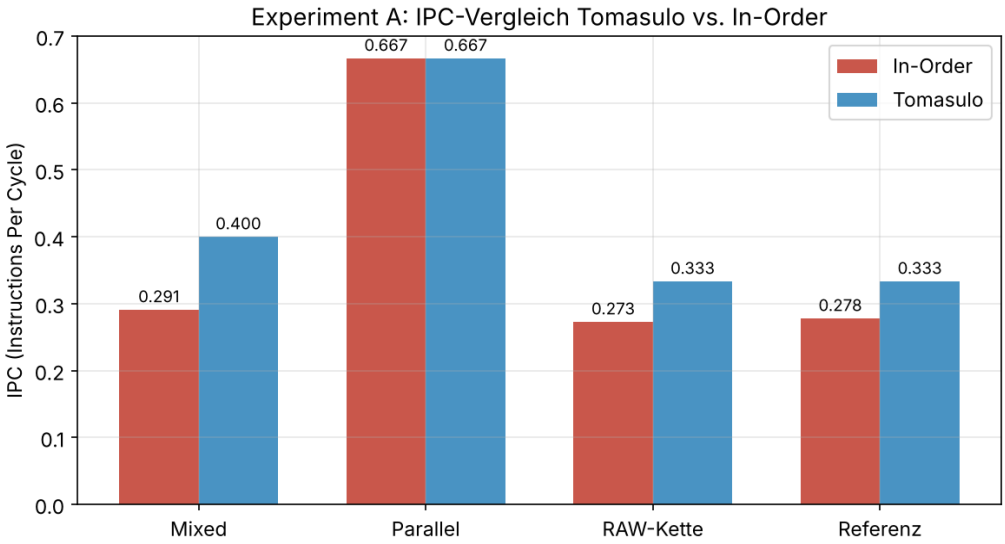


Abbildung 4: IPC-Vergleich Tomasulo vs. In-Order

Trace	Tom. Zyklen	IO Zyklen	Speedup	Tom. IPC
Mixed	40	55	1.38×	0.4000
Parallel	18	18	1.00×	0.6667
RAW-Kette	9	11	1.22×	0.3333
Referenz	15	18	1.20×	0.3333

Tabelle 2: Vergleich Tomasulo vs. In-Order-Pipeline

Diskussion: Der Mixed-Trace zeigt den höchsten Speedup (1.38×), da er viele Abhängigkeiten enthält, die Tomasulo durch Out-of-Order-Ausführung überlappen kann. Der Parallel-Trace zeigt keinen Speedup

(1.00×), weil die In-Order-Pipeline bei unabhängigen Instruktionen ebenso effizient ist — der Engpass liegt hier nicht bei Datenabhängigkeiten, sondern beim Single-Issue-Durchsatz und dem einzelnen CDB.

4.2 Experiment B: Variation der Reservierungsstationen

Untersucht wird der Einfluss der RS-Anzahl (1, 2, 3, 4, 6, 8 pro Funktionseinheit) auf IPC und strukturelle Stalls.

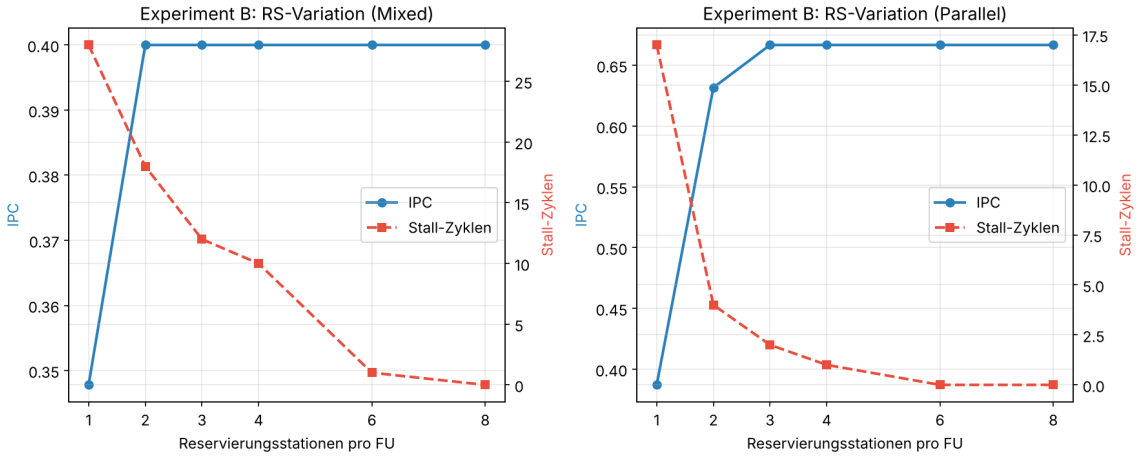


Abbildung 5: IPC und Stall-Zyklen in Abhängigkeit von der RS-Anzahl

Diskussion: Beim Mixed-Trace bleibt die Gesamtzyklenanzahl trotz sinkender Stall-Zyklen konstant bei 40 Zyklen. Dies zeigt, dass der kritische Pfad durch Datenabhängigkeiten (insbesondere die DIV-Kette) dominiert wird — strukturelle Stalls werden vollständig durch Ausführungslatenz überlagert. Beim Parallel-Trace führt die Erhöhung von RS=2 auf RS=4 zu einer messbaren Verbesserung (19→18 Zyklen), danach flacht die Kurve ab.

4.3 Experiment C: Variation der Latenzen

Die Latenzen von MUL und DIV werden systematisch erhöht, um den Einfluss auf Gesamtzyklen und Stall-Zyklen zu analysieren.

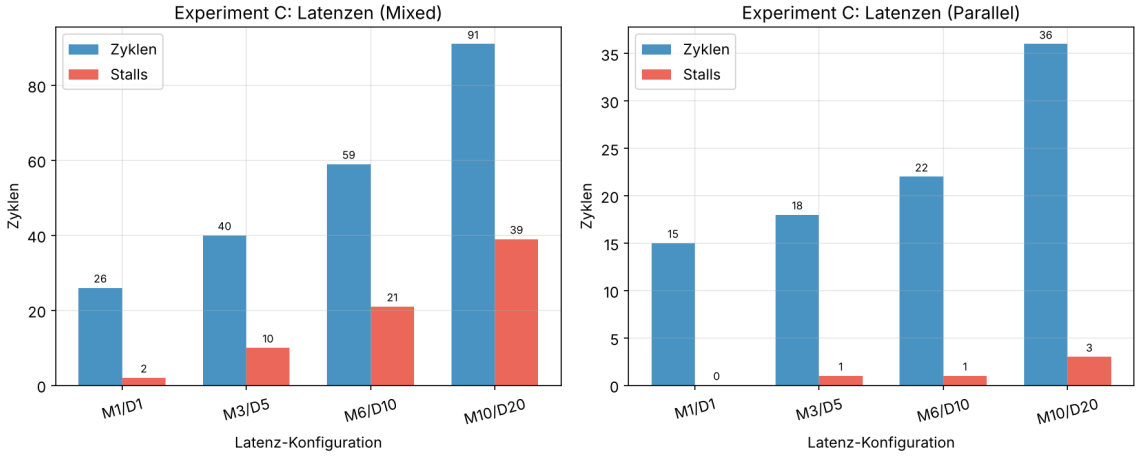


Abbildung 6: Gesamtzyklen und Stalls bei verschiedenen Latenzkonfigurationen

Diskussion: Höhere Latenzen vervielfachen die Gesamtausführungszeit nahezu proportional (Mixed-Trace: 26→91 Zyklen bei MUL=1/DIV=1 vs. MUL=10/DIV=20). Die Stall-Zyklen steigen ebenfalls stark, da die längere Belegung der RS zu mehr strukturellen Konflikten führt. Trotz steigender Latenzen bleibt Tomasulo effizienter als eine In-Order-Pipeline, da unabhängige Instruktionen weiterhin parallel ausgeführt werden können.

4.4 Experiment D: Analyse aufgelöster Abhängigkeiten

Analysiert wird der Anteil der durch Tomasulo transparent aufgelösten RAW-Abhängigkeiten im Vergleich zu den verbleibenden strukturellen Stalls.

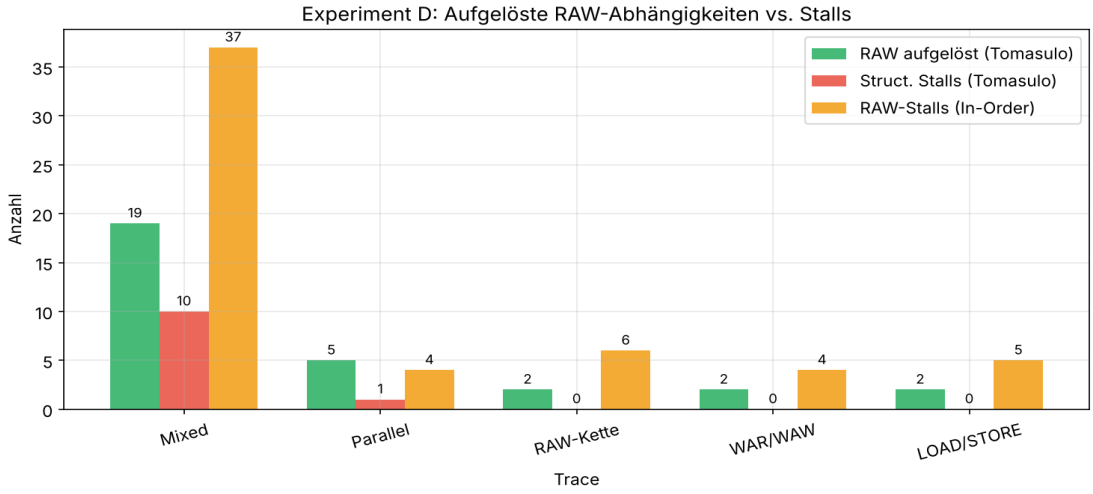


Abbildung 7: Aufgelöste RAW-Abhängigkeiten vs. Stalls

Trace	RAW aufgel.	Struct. Stalls	IO-Stalls	Anteil aufgel.
Mixed	19	10	37	65.5%
Parallel	5	1	4	83.3%
RAW-Kette	2	0	6	100%
WAR/WAW	2	0	4	100%
LOAD/STORE	2	0	5	100%

Tabelle 3: Analyse der aufgelösten RAW-Abhängigkeiten

Diskussion: Für die Validierungs-Traces (RAW-Kette, WAR/WAW, LOAD/STORE) löst Tomasulo 100% der RAW-Hazards transparent auf — es treten keinerlei strukturelle Stalls auf. Beim Mixed-Trace werden 65.5% aufgelöst; die verbleibenden 10 Stalls sind struktureller Natur (volle RS bei Standardkonfiguration mit 4 ALU-RS). Im Vergleich dazu benötigt die In-Order-Pipeline für denselben Trace 37 Stall-Zyklen — Tomasulo reduziert die Stalls also um 73%.

5. Zusammenfassung und Schlussfolgerungen

Die Implementierung des Tomasulo-Simulators konnte anhand von fünf Validierungsfällen und vier systematischen Experimenten erfolgreich verifiziert werden. Die wichtigsten Erkenntnisse:

1) Der Tomasulo-Algorithmus erzielt bei abhängigkeitsintensiven Traces einen Speedup von bis zu 1.38× gegenüber einer einfachen In-Order-Pipeline. Der Vorteil kommt durch die transparente Auflösung von RAW-Hazards über den CDB und die Möglichkeit, unabhängige Instruktionen parallel auszuführen.

2) WAR- und WAW-Scheinabhängigkeiten werden durch das implizite Register-Renaming der Reservierungsstationen vollständig eliminiert, ohne dass eine explizite Register-Allokationstabelle (RAT) benötigt wird.

3) Die Anzahl der Reservierungsstationen hat nur dann einen messbaren Einfluss auf die Gesamtperformance, wenn die Stall-Zyklen den kritischen Pfad verlängern. Bei abhängigkeitsdominanten Traces wird der Bottleneck durch Datenfluss, nicht durch Ressourcenknappheit bestimmt.

4) Höhere Ausführungslatenzen (MUL, DIV) wirken sich direkt auf die Gesamtzyklen aus und verstärken auch die strukturellen Konflikte, da RS länger belegt bleiben.

5) Der einzelne CDB kann bei stark parallelen Workloads zum Engpass werden, da pro Zyklus nur ein Ergebnis broadcast werden kann. In realer Hardware wird dies durch mehrere Busse oder Forwarding-Netzwerke adressiert.

6. Bedienungsanleitung

Voraussetzungen: Python 3.8 oder neuer. Keine externen Bibliotheken nötig.

Einzelnen Trace ausführen: python tomasulo.py traces/referenz.trace

Mit Konfigurationsparametern: python tomasulo.py traces/experiment_mixed.trace --alu-rs 8 --ls-rs 4 --lat-mul 6 --lat-div 10

Vergleich mit In-Order: python tomasulo.py traces/experiment_mixed.trace --compare

Alle Experimente: python run_experiments.py